

PRACTICAL – 5

Implementation of Policy Iteration Algorithm in Maze Problem

Aim

To implement the **Policy Iteration algorithm** to find the optimal policy for solving a Maze problem using Reinforcement Learning and to execute the implementation in Google Colab.

Code:

```
# Policy Iteration Algorithm for Maze Problem (Single Cell Code)
```

```
import numpy as np
```

```
# Define Maze Environment
```

```
# 0 = normal state
```

```
# -1 = obstacle
```

```
# +1 = goal state
```

```
maze = np.array([
```

```
    [0, 0, 0, 0],
```

```
    [0, -1, 0, -1],
```

```
    [0, 0, 0, -1],
```

```
    [-1, 0, 0, 1]
```

```
])
```

```
rows, cols = maze.shape
```

```
# Parameters
```

```
gamma = 0.9
```

```
theta = 0.0001
```

```
# Actions: Up, Down, Left, Right
```

```
actions = {
```

```
    0: (-1, 0), # Up
```

```
    1: (1, 0), # Down
```

```
    2: (0, -1), # Left
```

```
    3: (0, 1) # Right
```

```
}
```

```
action_symbols = {
```

```
    0: "↑",
```

```
    1: "↓",
```

```
    2: "←",
```

```
    3: "→"
```

```
}
```

```
# Initialize Value Function and Random Policy
```

```
V = np.zeros((rows, cols))
```

```
policy = np.random.choice(list(actions.keys()), size=(rows, cols))
```

```
# Policy Evaluation Function
```

```
def policy_evaluation(policy, V):
```

```
    while True:
```

```

delta = 0

for i in range(rows):
    for j in range(cols):

        if maze[i, j] == -1 or maze[i, j] == 1:
            continue

        action = policy[i, j]
        di, dj = actions[action]
        ni, nj = i + di, j + dj

        if 0 <= ni < rows and 0 <= nj < cols and maze[ni, nj] != -1:
            reward = maze[ni, nj]
            new_value = reward + gamma * V[ni, nj]
        else:
            new_value = gamma * V[i, j]

        delta = max(delta, abs(new_value - V[i, j]))
        V[i, j] = new_value

    if delta < theta:
        break

return V

```

Policy Improvement Function

```

def policy_improvement(V, policy):
    stable = True

    for i in range(rows):
        for j in range(cols):

            if maze[i, j] == -1 or maze[i, j] == 1:
                continue

            old_action = policy[i, j]
            action_values = []

            for action in actions:
                di, dj = actions[action]
                ni, nj = i + di, j + dj

                if 0 <= ni < rows and 0 <= nj < cols and maze[ni, nj] != -1:
                    reward = maze[ni, nj]
                    value = reward + gamma * V[ni, nj]
                else:
                    value = -999 # invalid move penalty

                action_values.append(value)

            best_action = np.argmax(action_values)
            policy[i, j] = best_action

```

```

        if old_action != best_action:
            stable = False

    return policy, stable

# Policy Iteration Loop
iteration = 0
while True:
    iteration += 1
    V = policy_evaluation(policy, V)
    policy, stable = policy_improvement(V, policy)

    if stable:
        break

# Print Results
print("Maze:")
print(maze)

print("\nOptimal Value Function:")
print(np.round(V, 2))

print("\nOptimal Policy (Directions):")
policy_display = []

```

```
for i in range(rows):  
    row = []  
    for j in range(cols):  
        if maze[i, j] == -1:  
            row.append("X")  
        elif maze[i, j] == 1:  
            row.append("G")  
        else:  
            row.append(action_symbols[policy[i, j]])  
    policy_display.append(row)  
  
for row in policy_display:  
    print(row)  
  
print("\nTotal Iterations:", iteration)
```